Copyright All rights reserved

06. 제어문:조건문
(Control Flow – Conditional Statements)

Heesung Oh
https://github.com/3dapi

Heesung Oh
Copyright All rights reserved

- scanf
  - 표준 입력(키보드)에서 값을 읽어 변수에 저장
  - 변수의 주소 사용
  - 하나의 함수 여러 자료형 입력 처리

```
scanf("형식지정자", 변수의 주소);
```

```
long long money = 0;
scanf("%lld", &money); // 변수의 주소로 값을 입력
```

6.1.2 scanf 함수 형식 지정자

Heesung Oh
Copyright All rights reserved

- **형식(서식) 지정자**

자료형	형식 지정자	예시 코드
int	**%d**	**scanf("%d", &n);**
long	%ld	scanf("%ld", &n);
long long	**%lld**	**scanf("%lld", &n);**
unsigned int	%u	scanf("%u", &n);
unsigned long long	%llu	scanf("%llu", &n);
float	%f	scanf("%f", &f);
double	%lf	scanf("%lf", &d);
char(문자 하나)	%c	scanf(" %c", &ch); (앞 공백은 버퍼 문제 방지)
char[] (문자열)	**%s**	**scanf("%s", str);** **(배열 이름은 문자열이 저장될 주소 반환 & 사용 안 함)**

Heesung Oh
Copyright All rights reserved

- **형식 지정자 패턴**

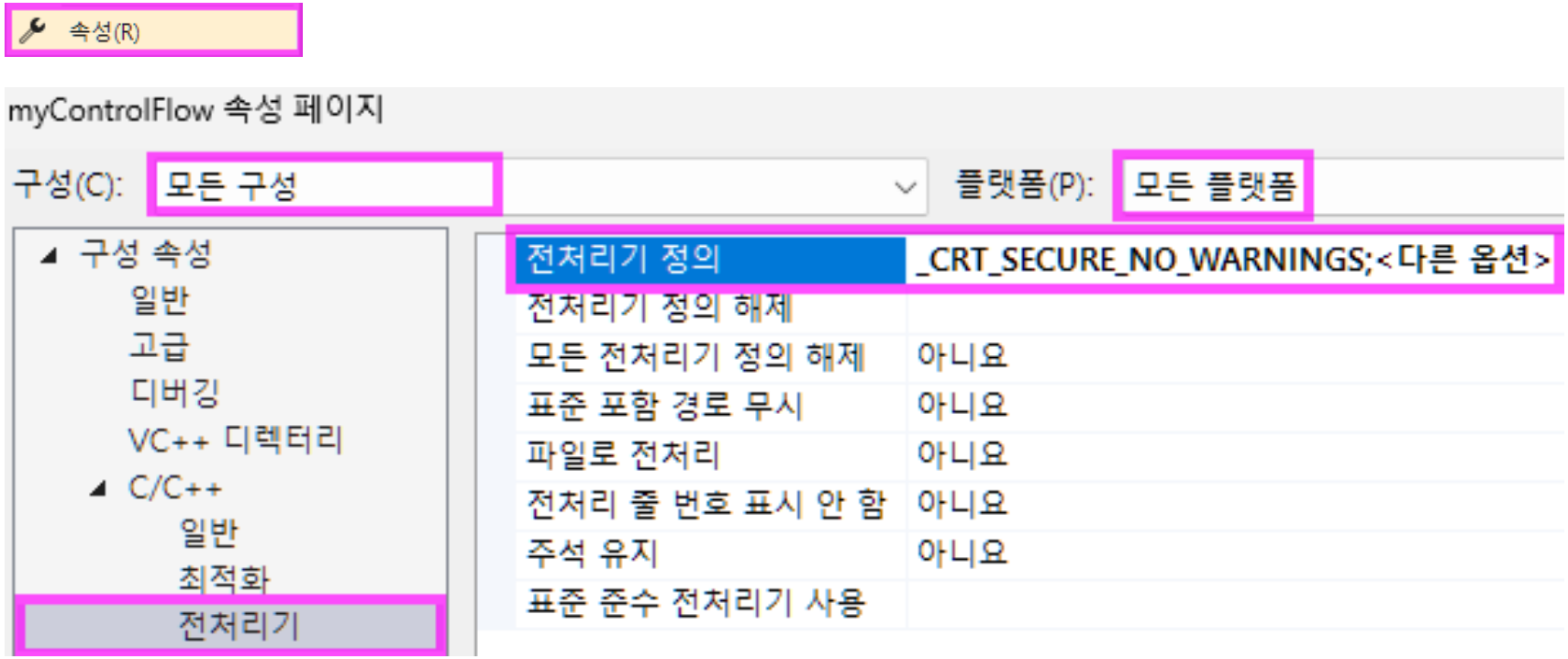
패턴	의미	용도
row: %d	"row: " 문자열 뒤의 정수 읽기	메타데이터 파싱
%63s	**공백 전까지 최대 63문자 읽기**	**버퍼 오버플로 방지**
%[characters]	지정한 문자 집합에 속하는 문자만 읽기	%[A-Za-z0-9_] : 영문자, 숫자, _만 읽기
%[^character]	특정 문자가 나오기 전까지 읽기	CSV 필드 파싱
%127[^character]	최대 127자까지 특정 문자 전까지 읽기	버퍼 오버플로 방지
%*s	**공백 전까지 읽되 저장하지 않음**	**불필요한 컬럼 건너뛰기**
%*[^character]	**특정 문자 전까지 읽되 저장하지 않음**	**CSV 컬럼 스킵**

Heesung Oh
Copyright All rights reserved

- **Unsafe error**

```
error C4996: 'scanf': This function or variable may be unsafe.
Consider using scanf_s instead. To disable deprecation,
use  _CRT_SECURE_NO_WARNINGS. See online help for details.
```

- **MSVC 설정**



6.1.4 scanf – 여러 개의 값을 한 번에 입력 받기

Heesung Oh
Copyright All rights reserved

- scanf 공백 사용

```
scanf("%d %d %d", &a, &b, &c);
```

```
#include <stdio.h>

int main(void)
{
    int x = 0, y = 0;
    printf("두 정수를 입력하세요: ");
    scanf("%d %d", &x, &y);
    printf("합: %d\n", x + y);
    return 0;
}
```

6.1.5 문자를 입력 받을 때 주의할 점 (%c)

Heesung Oh
Copyright All rights reserved

- scanf – 문자 입력: 버퍼의 '\n' 이 공백 사용

```
char ch;
scanf("%d", &n);
scanf("%c", &ch); // 버퍼의 '\n'이 입력됨
```

- 100% 해결이 안됨 → fgets 사용

```
scanf(" %c", &ch);
```

6.1.6 문자열 입력 (%s)

Heesung Oh
Copyright All rights reserved

- scanf
  - 문자열: %s 형식 지정자
  - 공백 전까지 문자열 저장

```
char name[64]={0, };
scanf("%s", name);
```

```
#include <stdio.h>

int main(void)
{
    char message[64]={0, };
    printf("메시지를 입력하세요: ");
    scanf("%s", message);
    printf("메시지 출력: %s\n", message);
    return 0;
}
```

6.1.6 문자열 입력 (%s) → fgets

Heesung Oh
Copyright All rights reserved

- fgets
  - 길이만큼 입력
  - 공백 포함 '\n' 까지 저장. 제거 필요

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char name[64]={0,};
    fgets(name, sizeof(name), stdin);
    // 문자열 끝의 '\n' 제거
    name[strcspn(name, "\n")] = '\0';
    printf("name: %s\n", name);
    return 0;
}
```

6.1.7 scanf 반환값

Heesung Oh
Copyright All rights reserved

- 입력 성공한 항목 개수 반환
  - Ex) "%d %d %d" → 모두 성공 시 3 반환

```
#include <stdio.h>

int main(void)
{
    int n = -999999; // scanf 실패하면 -999999 값이 유지됨.
    int ret = scanf("%d", &n);
    printf("scanf 결과: %d, 읽은 값: %d\n", ret, n);

    n = -999999; // scanf 실패하면 -999999 값이 유지됨.
    ret = scanf("%d", &n);
    printf("scanf 결과: %d, 읽은 값: %d\n", ret, n);

    return 0;
}
```

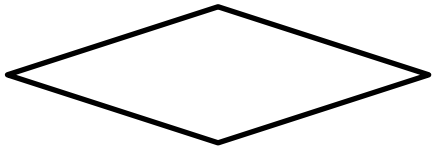
6.1.8 입력 관련 종합 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    // 문자열을 안 배웠지만 미리 알아 둔다.
    char player_name[64]= {0, };
    int  player_level = -999999;
    int  player_hp = -999999;
    long long player_gold = -999999;

    printf("플레이어의 이름을 입력하세요 :");   scanf("%s", player_name);
    printf("플레이어의 레벨을 입력하세요:");    scanf("%d", &player_level);
    printf("플레이어의 체력을 입력하세요 :");   scanf("%d", &player_hp);
    printf("플레이어의 골드를 입력하세요 :");   scanf("%lld", &player_gold);

    printf("========================\n");
    printf("플레이어 정보\n");
    printf("이름: %s\n", player_name);
    printf("레벨: %d\n", player_level);
    printf("체력: %d\n", player_hp);
    printf("골드: %lld\n", player_gold);

    return 0;
}
```

Heesung Oh
Copyright All rights reserved

- **if 문: 조건이 참일 때 실행되는 분기문**



```
// 실행문 1줄
if (조건식)
    실행문1;
else
    실행문2
```

```
// 코드 블록
if (조건식)
{
    실행문1;
}
else
{
    실행문2;
}
```

```
// 실행문 1줄
if (조건식1)
    실행문1;
else if (조건식2)
    실행문2;
else if (조건식3)
    실행문3;
else
    실행문4;
```

```
// 코드 블록
if (조건식1)
{
    실행문1;
}
else if (조건식2)
{
    실행문2;
}
else if (조건식3)
{
    실행문3;
}
else
{
    실행문4;
}
```

```
// 코드 블록
if (조건식1)
{
    실행문1;
}
else
{
    if (조건식2)
    {
        실행문2;
    }
    else
    {
        if (조건식3)
        {
            실행문3;
        }
        else
        {
            실행문4;
        }
    }
}
```

6.2.3 조건식의 평가 규칙과 암시적 조건 검사

Heesung Oh
Copyright All rights reserved

- 조건식의 평가 규칙
  - 0은 거짓
  - 0이 아닌 모든 값은 참, -1도 참
  - 포인터 NULL은 거짓
  - 널 문자 `'\0'` 은 거짓
  - 연산 결과가 참/거짓으로 해석 가능한 식이면 모두 사용 가능

6.2.3 조건식의 평가 규칙과 암시적 조건 검사

Heesung Oh
Copyright All rights reserved

- 암시적 조건 검사
  - 값을 직접 비교하지 않고 참/거짓으로 해석해서 판단

```
if (0 != n)
```

```
if (n)
```

```
// 숫자
if (0 == n)
// 문자
if ('\0' == a)

// 포인터
if (NULL == p)
```

```
// 숫자, 문자, bool,
// 포인터 구분 없이 사용
if (!n)
```

6.2.4 조건문 사용 시 주의 사항

Heesung Oh
Copyright All rights reserved

- 초기화 안 한 지역 변수

```
void func()
{
    int n; // 초기화되지 않음
    if (n > 0)
        printf("양수\n");
```

```
void func()
{
    int n = 0; // 반드시 초기화
    if (n > 0)
        printf("양수\n");
```

- 비교 연산자(==) 대신 대입 연산자(=) 사용

```
if (n = 10)
```

```
if (10 == n)
```

- else 연결 문제

```
if (hp > 0)
    if (attackState != 0)
        printf("공격 상태\n");
else
    printf("죽음 상태\n");
```

```
if (hp > 0)
{
    if (attackState != 0)
        printf("공격 상태\n");
    else
        printf("죽음 상태\n");
}
```

6.2.4 조건문 사용 시 주의 사항

Heesung Oh
Copyright All rights reserved

- **조건 순서 오류**

```c
if (score >= 70)
    printf("C\n");
else if (score >= 80) // 절대 실행 안됨
    printf("B\n");
else if (score >= 90)
    printf("A\n");
```

```c
if (score >= 90)
    printf("A\n");
else if (score >= 80)
    printf("B\n");
else if (score >= 70)
    printf("C\n");
```

- **범위 비교 오류**

```c
if (score >= 0 || score <= 100)// 항상 true
```

```c
if (score >= 0 && score <= 100)
```

- **복잡한 조건식**

```c
if (a > 0 && b < 10 || c == 3 && !(d != 5))
```

```c
int validRange = (a > 0 && b < 10);
int isSpecial  = (c == 3 && d == 5);

if (validRange || isSpecial)
```

6.2.4 조건문 사용 시 주의 사항

Heesung Oh
Copyright All rights reserved

- **비 효율 중복 조건**

```
if (n > 10)
    printf("10보다 큼\n");
else if (n > 5)
    printf("5보다 큼\n");
else if (n > 10)   // 중복 검사
    printf("???\n");
```

- **부정 조건(!) 지나치게 남용 오류**

```
if (!flag)
if (!count)
if (!p)
```

```
if (0 == flag)
if (0 == count)
if (NULL== p)
```

- **중첩이 깊은 조건문 → early return**

```
if (조건A)
{
    if (조건B)
    {
        if (조건C)
        {
            // 실행 코드
        }
    }
}
```

```
// 함수가 void
if (!조건A)
    return;
if (!조건B)
    return;
if (!조건C)
    return;

if (!조건A)
    return 실패에 대한 약속된 값;
if (!조건B)
    return 실패에 대한 약속된 값;
if (!조건C)
    return 실패에 대한 약속된 값;
```

6.3 switch-case

Heesung Oh
Copyright All rights reserved

- Switch-case
  - 정수 값 기준 여러 경우 분기 처리
  - 표현식 값 계산 후 일치 case 레이블 실행

- 동작 방식
  - 표현식 값 계산
  - 일치 case 실행
  - break → switch 종료
  - default → 일치 없음 처리

- 특징
  - 정수형(int, char, enum) 사용
  - case 값 상수만 가능
  - break 없으면 다음 case 계속 실행

- 주의사항
  - break 누락 시 fall-through 발생
  - default 생략 가능하지만 권장

6.3.1 switch-case 기본 형식

Heesung Oh
Copyright All rights reserved

```
switch (표현식)
{
    case 값1:
        실행문1;
        break;

    case 값2:
        실행문2;
        break;

    case 값3:
        실행문3;
        break;

    default:
        실행문4;
        break;
}
```

```
switch (표현식)
{
    case 값1:
    {
        실행문1;
        break;
    }
    case 값2:
    {
        실행문2;
        break;
    }
    case 값3:
    {
        실행문3;
        break;
    }
    default:
    {
        실행문4;
        break;
    }
}
```

6.3.2 switch-case 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    int menu = 0;

    printf("메뉴를 선택하세요 (1: 시작, 2: 플레이, 3: 종료): ");
    scanf("%d", &menu);

    switch (menu)
    {
        case 1:
            printf("게임을 시작합니다.\n");
            break;
        case 2:
            printf("게임을 플레이합니다.\n");
            break;
        case 3:
            printf("게임을 종료합니다.\n");
            break;
        default:
            printf("잘못된 입력입니다.\n");
            break;
    }

    return 0;
}
```

6.3.3 fall-through

Heesung Oh
Copyright All rights reserved

- fall-through
  - break 생략 시 다음 case 연속 실행
  - 의도적 사용 가능. 대부분 실수로 발생

```
int n = 1;

switch (n)
{
    case 1:
        printf("1입니다.\n");   // 실행
    case 2:
        printf("2입니다.\n");   // break가 없으므로 실행됨
    case 3:
        printf("3입니다.\n");   // break가 없으므로 실행됨
}

// 출력 결과:
// 1입니다.
// 2입니다.
// 3입니다.
```

6.3.4 fall-through 의도적 사용

Heesung Oh
Copyright All rights reserved

- fall-through 사용 예: 소문자, 대문자 처리

```
switch (ch)
{
    case 'a':
    case 'A':
        printf("A 선택\n");
        break;
    case 'b':
    case 'B':
        printf("B 선택\n");
        break;
    default:
        printf("기타 문자\n");
        break;
}
```

6.3.5 switch 문에서 사용할 수 있는 값의 제한

Heesung Oh
Copyright All rights reserved

- switch 값 제한
  - 표현식, 정수형만 사용
- case 규칙
  - 상수만 사용, 연산 상수 가능, 변수 사용 불가, 중복 불가
- 특징
  - jump table 기반 분기, 정수값 → 직접 분기 가능
- 비교
  - if-else, 순차 비교
- switch
  - 직접 분기, 한 줄 정리
- 분기 많고 정수 기반 → switch 사용

6.3.6 enum과 switch 문

Heesung Oh
Copyright All rights reserved

- enum과 switch
  - 의미 있는 값 집합 정의
  - 정수 대신 이름 사용
- 장점
  - 코드 의미 명확.
  - 가독성 향상. 실수 감소
- 활용
  - 상태 분기. 명령 처리. 단계 구분
- 사용 분야
  - 게임
    - 캐릭터 상태, AI, 아이템
  - 네트워크
    - 패킷 종류 구분
  - 시스템
    - 상태, 오류 코드

```
enum State
{
    STATE_IDLE,
    STATE_MOVE,
    STATE_ATTACK,
    STATE_DEATH
};

enum State heroState = STATE_MOVE;
switch (heroState)
{
    case STATE_IDLE:
        printf("대기\n");
        break;
    case STATE_MOVE:
        printf("이동\n");
        break;
    case STATE_ATTACK:
        printf("공격\n");
        break;
    case STATE_DEATH:
        printf("사망\n");
        break;
}
```

6.3.7 switch 문 사용 시 주의 사항

Heesung Oh
Copyright All rights reserved

- 의도하지 않은 fall-through

```
switch (n)
{
    case 1:
        printf("1\n");   // break 없음
→ 다음 case 실행됨
    case 2:
        printf("2\n");
        break;
}
```

- default 미 작성 문제

```
switch (cmd)
{
    case 1: ...; break;
    case 2: ...; break;
    // default 없음 → 예상치 못한 입력
처리 불가
}
```

6.3.7 switch 문 사용 시 주의 사항

Heesung Oh
Copyright All rights reserved

- **case 값 중복**

```
case 1:
case 1:   // 중복 → 컴파일 오류
```

```
enum Command
{
    CMD_MOVE   = 100,
    CMD_ATTACK = 100,   // 서로 다른 이름이지만 동일한 값
};
```

```
switch (cmd)
{
    case CMD_MOVE:
        ...
        break;

    case CMD_ATTACK:   // CMD_MOVE와 동일한 값 → 컴파일 오류
        ...
        break;
```

6.3.7 switch 문 사용 시 주의 사항

Heesung Oh
Copyright All rights reserved

- **변수의 범위(scope) 문제**

```
switch (n)
{
    case 1:
        int x = 10;      // 선언됨
    case 2:
        printf("%d\n", x);  // 오류: x는 이 영역에서 보이지 않음
        break;
}
```

- **중괄호 추가**

```
switch (n)
{
    case 1:
    {
        int x = 10;
    }
    case 2:
    {
        printf("%d\n", x);
    }
}
```

```
switch (n)
{
    case 1:
    {
        int x = 10;
        printf("%d\n", x);
        break;
    }
    case 2:
    {
        printf("case 2\n");
        break;
    }
}
```

6.3.8 switch 문자 입력 + enum + fall-through

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

enum Command
{
    CMD_UNKNOWN  = -1,
    CMD_MOVE     =  1,
    CMD_ATTACK     ,
    CMD_QUIT     ,
};
```

```c
enum Command GetCommand(char key)
{
    switch(key)
    {
        case 'g':
        case 'G':
            return CMD_MOVE;
        case 'a':
        case 'A':
            return CMD_ATTACK;
        case 'q':
        case 'Q':
            return CMD_QUIT;
        default:
            return CMD_UNKNOWN;
    }
}
```

```c
int main(void)
{
    char input = 0;

    printf("명령 입력 (G/A/Q): ");
    scanf(" %c", &input);

    enum Command cmd = GetCommand(input);
    switch(cmd)
    {
        case CMD_MOVE:
            printf("이동 명령\n");
            break;

        case CMD_ATTACK:
            printf("공격 명령\n");
            break;

        case CMD_QUIT:
            printf("종료 명령\n");
            break;

        default:
            printf("알 수 없는 명령입니다.\n");
            break;
    }

    return 0;
}
```

6.3.9 if - else 문 VS switch - case 선택

Heesung Oh
Copyright All rights reserved

- 단순 분기 → switch, 복잡 조건 → if-else

- switch
  - 정수 값 기준
  - 적합 상태, 메뉴 입력 명령 코드, 패킷 타입

- switch 부적합
  - 범위 비교(score 〉= 90)
  - 복합 조건(a 〉 b && c 〈 d)
  - 다중 변수 비교
  - 실수 값 조건
  - case에 사용할 값이 런타임에서 결정될 때

- if - else 사용
  - 복잡한 조건 처리
    다양한 비교 가능

학습 정리

Heesung Oh
Copyright All rights reserved

- scanf
  - %c → 공백, 개행 포함, %d → 정수 입력, %s → 공백 전까지 입력
  - fgets → 공백 포함 문자열, 반환값 → 입력 성공 개수
- if 문
  - 조건 기반 분기, if – else: 참 / 거짓 분기
- if 주의
  - 0 → 거짓, 0이 아닌 값 → 참, NULL, '\0' → 거짓
- if 실수
  - 변수 미초기화, = / == 혼동, 중괄호 생략 위험, 조건 복잡 → 가독성 저하
- If 개선 방법
  - 조건 분리, 중간 변수 사용, early return 가독성 향상

학습 정리

Heesung Oh
Copyright All rights reserved

- switch 문
  - 정수 값 기준 분기, case 단위 분기 처리, 구조 단순, 가독성 높음
- switch 주의
  - break 누락 → fall-through, 변수 선언 시 블록 필요
  - 정수형만 사용, case 값 상수, 중복 불가
- switch 특징
  - jump table 기반 분기, enum + switch
  - 의미 있는 값 정의, 상태, 명령 처리에 적합
- 선택 기준
  - 단순 정수 분기 → switch
  - 복잡 조건 처리 → if - else